

CSIS 2300 Introduction to Algorithms

Spring 2026 – Homework 4 – Decrease & conquer; divide & conquer

Instructions: You do not need to typeset your solutions; however, your submission must be a single PDF file. The assignment is due by **8:59pm on Tuesday, February 10**. Remember that you may submit as many times as you wish before the deadline (we'll only grade the last one), but late submissions will receive no credit – so submit early, submit often!

Reminder: Exam 1 is on Wednesday, February 11 at 8am.

This assignment consists of problems numbered 1-6; be sure to look at the second page.

(1) Prove that the solution to the Josephus problem $J(k) = 1$ (that is, the last person standing is #1) whenever $k = 2^n$ for any positive integer n .

The argument here amounts to each round reducing to a problem of half the size, starting at person #1 again.

The easiest proof is probably inductive. Base case is $n = 0$ (i.e. $k = 1$), in which case there's only one person and the answer is trivial. The inductive step says that if it works for n (2^n people), then it works for $n + 1$ (2^{n+1} people). Consider a case with 2^{n+1} people. Since $n \geq 0$, this is an even number, and in any circle with an even number of people, one round eliminates all of the even-numbered people and leaves us back at person #1. This leaves a circle with 2^n people, starting at person #1; by the I.H., this ends with person #1 as the last survivor. ■

(2) Imagine a pile of n large stone discs of distinct sizes as in Towers of Hanoi, but these discs are in a random order, with larger ones possibly on top of smaller ones. (Don't think too hard about stability. :P) You have a machine that can `grab` a disc and every disc above it in the pile and flip the entire grabbed stack. (For example, given 3 6 1 2 4 5, with 3 on top, a call to `grab(4)` would grab 3 6 1 2 and flip that group, producing 2 1 6 3 4 5.)

Design an algorithm that uses calls to `grab` and results in the pile being in sorted order with the smallest disc on top. What is your algorithm's runtime, assuming `grab` is the critical operation? (i.e. How many calls to `grab` do you need, asymptotically?)

A straightforward approach uses $2n$ calls to `grab`. For every disc i from n down to 1, do the following: first, grab the pile down to wherever disc i is located and flip it so that disc i is on top of the pile. Then `grab(i)`, resulting in disc i being in position i . (This is essentially a selection sort.)

(3) Demonstrate the execution of quicksort on the letters of the word PROSECUTIONS, using Hoare's partition method. You should show the new order after each partition. Also, use subscripts to distinguish multiple copies of the same letter. (i.e. Your initial positions should be written as $PRO_1S_1ECUTIO_2NS_2$.) Note that your sort might not be stable.

P is the initial pivot.

$PRO_1S_1ECUTIO_2NS_2$ – initial position. Left pointer will stop at R ; right will move to N .

$PNO_1S_1ECUTIO_2RS_2$ – left pointer moves up to S_1 ; right stops at O_2

$PNO_1O_2ECUTIS_1RS_2$ – left pointer moves up to U ; right stops at I

$PNO_1O_2ECITUS_1RS_2$ – left pointer moves up to U ; right stops at I

$PNO_1O_2ECITUS_1RS_2$ – left pointer moves up to T ; right stops at I (yes, again!)

$PNO_1O_2ECTIUS_1RS_2$ – They passed each other! Switch back.

$PNO_1O_2ECITUS_1RS_2$ – Finally, swap the pivot with the location of the “right” pointer.

$INO_1O_2ECPTUS_1RS_2$ – Left-side recursive call (level 1); I is the pivot

$INO_1O_2ECPTUS_1RS_2$ – Left pointer stops at N ; right stops at C

$ICO_1O_2ENPTUS_1RS_2$ – Left pointer stops at O_1 ; right stops at E

$ICEO_2O_1NPTUS_1RS_2$ – Left pointer stops at O_2 ; right stops at E

$ICO_2EO_1NPTUS_1RS_2$ – Overshot, put it back.

$ICEO_2O_1NPTUS_1RS_2$ – Move the pivot

$ECIO_2O_1NPTUS_1RS_2$ – Left-left recursion (level 2); E is the pivot

$ECIO_2O_1NPTUS_1RS_2$ – Pointers overshoot immediately – C & E swap and swap back.

$CEIO_2O_1NPTUS_1RS_2$ – We move the pivot and then make base-case calls of size 1 & 0.

$CEIO_2O_1NPTUS_1RS_2$ – Left-right recursion (level 2); O_2 is the pivot

$CEIO_2O_1NPTUS_1RS_2$ – Left pointer stops at O_1 ($O \geq O$); right pointer stops at N

$CEIO_2NO_1PTUS_1RS_2$ – Both pointers immediately stop again – we swap and swap back.

$CEINO_2O_1PTUS_1RS_2$ – We move the pivot and then make two base-case recursive calls.

$CEINO_2O_1PTUS_1RS_2$ – Right-side recursive call (level 1); T is the pivot

$CEINO_2O_1PTUS_1RS_2$ – Both pointers stop immediately (U and S_2)

$CEINO_2O_1PTS_2S_1RU$ – Left pointer runs all the way to U ; right stops immediately at R

$CEINO_2O_1PTS_2S_1UR$ – Overshot, so swap back

$CEINO_2O_1PTS_2S_1RU$ – Move the pivot into place

$CEINO_2O_1PRS_2S_1TU$ – Right-left recursion (level 2); R is the pivot

From here on out, nothing moves except for some swaps and immediate swap backs.

Observe that the final sort is demonstrably not stable – both pairs of duplicate letters end up in the reverse of the order they started in.

(4) Find the order of growth for solutions to these recurrences. Show your work.

(a) $T(n) = 4T(n/2) + n, T(1) = 1$

(b) $T(n) = 4T(n/2) + n^2, T(1) = 1$

(c) $T(n) = 4T(n/2) + n^3, T(1) = 1$

The Master Theorem will work on all three of these.

(a) $a = 4, b = 2, d = 1; a > b^d$, so we're $\Theta(n^{\log_b a})$, which in this case means $\Theta(n^2)$

(b) $a = 4, b = 2, d = 2; a = b^d$, so we're $\Theta(n^d \log n)$, which in this case means $\Theta(n^2 \log n)$

(c) $a = 4, b = 2, d = 3; a < b^d$, so we're $\Theta(n^d)$, which in this case means $\Theta(n^3)$

(5) You are given a jumbled-up collection of n bolts of different widths and n corresponding nuts. You can try a nut/bolt combination and determine if the nut is smaller than the bolt, larger than the bolt, or matches precisely, but you may not compare two nuts, nor two bolts. Design an algorithm to match all bolts with their corresponding nuts that has average-case efficiency of $O(n \log n)$.

The idea here is a simultaneous / parallel quicksort. Pick any nut and try every bolt against it, separating into piles of smaller and larger, plus the one bolt that matches. Now, take that matching bolt and try every other nut against it, again separating into piles of larger and smaller. Now, the smaller nuts and bolts go together, as do the larger nuts and bolts. So we can repeat the process on these two sets. Since this really is just two quicksorts, it has the same average-case efficiency as quicksort - $O(n \log n)$.

(6) Recall the procedure for multiplying a pair of 2×2 matrices:

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{bmatrix}$$

One can build a divide-and-conquer algorithm for multiplying larger matrices together using the same idea. Assume for simplicity that A and B are $n \times n$ matrices where $n = 2^k$ for some positive integer k . We can split each of A and B into four quadrants, where each quadrant is an $(n/2) \times (n/2)$ matrix, and it turns out that:

$$AB = \begin{bmatrix} A_{UL} & A_{UR} \\ A_{LL} & A_{LR} \end{bmatrix} \begin{bmatrix} B_{UL} & B_{UR} \\ B_{LL} & B_{LR} \end{bmatrix}$$

Multiplying the quadrants (which are themselves matrices) can then be handled recursively.

- How many (recursive) matrix multiplications are needed for this technique?
- How many matrix additions are needed for this technique? What is the asymptotic runtime of performing these matrix additions?
- Use your answers from (a) and (b) to write the recurrence relation governing this recursive matrix multiplication technique. Then use the Master Theorem to find its closed-form asymptotic runtime.
- It is possible to accomplish this task with less multiplication and more addition. Specifically, **seven** multiplications and 18 additions/subtractions are needed. Write the recurrence relation for this new technique, and use the Master Theorem to find its closed-form asymptotic runtime.

- We do eight multiplications: Two rows $\times$ two columns $\times$ two matrix multiplications each time.
- We add a pair of $(n/2) \times (n/2)$ matrices a total of four times. Each one requires $O(n^2)$ time (as they require a quadratic number of scalar additions).
- $T(n) = 8T(n/2) + O(n^2)$; $a = 8, b = 2, d = 2$. Since $a > b^d$, the time is $O(n^{\log_b a})$, which in this case gives us $O(n^3)$ time.
- As long as the number of smaller matrix additions is a constant and not dependent on n , the non-recursive part of this will still be $O(n^2)$. So the recurrence relation becomes $T(n) = 7T(n/2) + O(n^2)$. a is still larger than b^d , but now $\log_b a = \log_2 7$, which is about 2.81. So the Strassen algorithm is $O(n^{2.81})$ – which may not seem like a huge improvement, but when matrix multiplication is as common as it is, it becomes a big deal.